

08 — Operatoren

Werkzeuge für Berechnungen und Entscheidungen

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Wozu Operatoren?
2. Arithmetische Operatoren
3. Vergleichsoperatoren
4. Logische Operatoren (and, or, not)
5. Identität (is) vs. Gleichheit (==)
6. Zugehörigkeit (in, not in)
7. Mengenoperationen auf Sets (|, &, -, ^)

Lernziel: Operatoren als Bausteine für Geschäftslogik nutzen – Preise rechnen, Bedingungen kombinieren, Listen prüfen.

Wie wir heute arbeiten: Nach jedem Konzept zeigt die Folie *Live-Demo* (ich im Terminal) und *Sofort ausprobieren* (Sie im Notebook 08).

Wozu Operatoren?

Ohne Operatoren

```
def brutto():  
    return 100.0    # Hardcodiert -- jede Aenderung muss von Hand
```

Mit Operatoren

```
netto = 89.95  
mwst  = 0.19  
brutto = netto * (1 + mwst)           # 107.04  
versandfrei = brutto >= 50           # True
```

Operatoren verknuepfen Werte zu neuen Werten - *die Sprache der Ge-schaeftslogik.*

Arithmetische Operatoren

Operator	Bedeutung	Beispiel
+	Addition	$89.95 + 109.00$
-	Subtraktion	$109.00 - 89.95$
*	Multiplikation	$89.95 * 3$
/	Division (float)	$100 / 3 \rightarrow 33.33\dots$
//	Ganzzahl-Division	$100 // 3 \rightarrow 33$
%	Modulo (Rest)	$100 \% 3 \rightarrow 1$
**	Potenz	$2 ** 10 \rightarrow 1024$

`brutto = netto * (1 + mwst)`

► **Live-Demo** - 01_arithmetik_preise.py

✎ **Sofort ausprobieren** - Notebook 08, Kap. 1: Plus, Minus, Mal, Geteilt, Modulo, Potenz (sechs kleine Aufgaben)

Spezialfälle: / vs. // vs. %

```
print(7 / 2)          # 3.5          -- echte Division, immer float
print(7 // 2)         # 3            -- Ganzzahl-Division (abrunden)
print(7 % 2)          # 1            -- Rest
print(2 ** 10)        # 1024        -- Potenz
```

Wofür ist % praktisch?

```
artikel_pro_karton = 12
bestellte_artikel   = 47
volle_kartons        = bestellte_artikel // artikel_pro_karton #
einzelartikel_rest   = bestellte_artikel % artikel_pro_karton  #
```

Faustregel: / ergibt immer float, // immer int, % gibt den Rest.

Vergleichsoperatoren

Operator	Bedeutung	Beispiel
<code>==</code>	gleich	<code>bestand == 0</code>
<code>!=</code>	ungleich	<code>kunde != "gast"</code>
<code><</code>	kleiner	<code>bestand < 5</code>
<code>></code>	groesser	<code>betrag > 50</code>
<code><=</code>	kleiner-gleich	<code>alter <= 17</code>
<code>>=</code>	groesser-gleich	<code>betrag >= 50</code>

- Vergleichsoperatoren liefern **immer** True oder False
- Ergebnistyp: **bool**

Vergleichsoperatoren

► **Live-Demo** - 02_vergleich_bestand.py

✎ **Sofort ausprobieren** - Notebook 08, Kap. 2: ==, !=, <, >, <=, >=

Achtung: = vs. ==

```
preis = 89.95           # ZUWEISUNG -- "Setze preis auf 89.95"  
preis == 89.95          # VERGLEICH -- "Ist preis gleich 89.95?"
```

	=	==
Was tut es?	weist Wert zu	vergleicht Werte
Ergebnis	nichts (Seiteneffekt)	True / False
Wo?	links eine Variable	irgendwo in einem Ausdruck

Fehlerklassiker: `if x = 5` ist Syntax-Fehler – gemeint ist `if x == 5`.

Logische Operatoren - and, or, not

```
darf_rabatt = stammkunde and bestellsumme >= 50  
versandfrei = bestellsumme >= 50 or premium_mitglied  
nicht_storniert = not storniert
```

Wahrheitstabelle (Kurzform)

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

Logische Operatoren - and, or, not

► **Live-Demo** - 03_logik_versandfrei.py

✎ **Sofort ausprobieren** - Notebook 08, Kap. 3: Party-Bedingung (and), Museum kostenlos (or), not-Operator

Vorrangregeln (Operator Precedence)

Python wertet **wie in Mathe** aus:

1. () – Klammern zuerst
2. ** – Potenz
3. *, /, //, %
4. +, -
5. Vergleiche (==, <, ...)
6. not
7. and
8. or

Vorrangregeln (Operator Precedence)

ohne Klammern -- riskant zu lesen

```
a = stammkunde or bestellsumme >= 50 and premium
```

mit Klammern -- klar:

```
a = stammkunde or (bestellsumme >= 50 and premium)
```

Faustregel: lieber **eine Klammer zu viel** als ein Bug, den man drei Tage sucht.

is vs. == - Identität vs. Gleichheit

```
liste_a = [1, 2, 3]
liste_b = liste_a           # gleicher "Briefumschlag"
liste_c = [1, 2, 3]        # zweiter Umschlag, gleicher Inhalt

print(liste_a == liste_b)  # True   (gleicher Inhalt)
print(liste_a is liste_b)  # True   (identisches Objekt)
print(liste_a == liste_c)  # True   (gleicher Inhalt)
print(liste_a is liste_c)  # False  (anderes Objekt!)
```

is vs. == - Identität vs. Gleichheit

- == fragt: **“Sind die Werte gleich?”** (fast immer das, was Sie wollen)
- is fragt: **“Ist es dasselbe Objekt im Speicher?”**

Praktischer Einsatz von is: nur fuer is None, is True, is False.

► **Live-Demo** - 05_identitaet_vs_gleichheit.py

in und not in - Zugehörigkeit

```
warenkorb = ["Eco-Sneaker", "Hemp-High"]
```

```
print("Eco-Sneaker" in warenkorb)           # True
```

```
print("Bambus-Boot" not in warenkorb)       # True
```

- Funktioniert mit **Listen, Tupeln, Sets, Dicts und Strings**
- Bei Strings prüft in Teil-Strings:

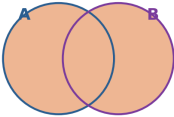
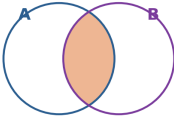
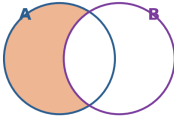
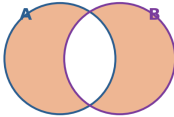
```
"sneaker" in "Eco-Sneaker".lower()         # True
```

► **Live-Demo** - 04_zugehoerigkeit_warenkorb.py

✎ **Sofort ausprobieren** - Notebook 08, Kap. 4: is-Operator, in mit Lieblingszahlen, not in mit Benutzernamen, pruefe_liste-Funktion

Mengenoperationen auf Sets

Ein **Set** ($\{ \dots \}$) ist eine *Menge* – Operatoren rechnen damit wie in der Mengenlehre:

Vereinigung · $A \mid B$	Schnittmenge · $A \& B$	Differenz · $A - B$	Symm. Differenz · $A \wedge B$
			
$A \cup B$	$A \cap B$	$A \setminus B$	$A \triangle B$
$A \mid B = A \cup B$	$A \& B = A \cap B$	$A - B = A \setminus B$	$A \wedge B = A \triangle B$
alles aus beiden	nur das Gemeinsame	nur in A, nicht B	in genau einem

Mengenoperationen - konkret

```
eco  = {"Anna", "Max", "Lea"}      # kauften Eco-Sneaker  
hemp = {"Max", "Tom"}              # kauften Hemp-High
```

```
eco | hemp      # {'Anna', 'Max', 'Lea', 'Tom'}  
eco & hemp      # {'Max'}  
eco - hemp      # {'Anna', 'Lea'}  
eco ^ hemp      # {'Anna', 'Lea', 'Tom'}
```

(Vgl. die Venn-Diagramme der vorigen Folie – eco entspricht A , hemp entspricht B .)

Faustregel: Duplikate weg? Zwei Listen vergleichen? Gemeinsames finden?
→ in ein `set()` packen und mit `|` `&` `-` `^` rechnen.

Cheat Card

Klasse	Operatoren
Arithmetik	+ - * / // % **
Vergleich	== != < > <= >=
Logik	and or not
Identität	is, is not
Zugehörigkeit	in, not in
Mengen (Sets)	& - ^
Zuweisung	=, +=, -=, *=, /=

Faustregel: Vergleichs- und Logikoperatoren liefern bool. Genau diese bool-Werte brauchen wir ab Notebook 09 fuer if/elif/else.

Ausblick: Notebook 09 - Verzweigungen

Operatoren liefern Wahrheitswerte – damit kann das Programm endlich **entscheiden**:

```
if bestellsumme >= 50 and stammkunde:  
    print("versandfrei")  
elif bestellsumme >= 50:  
    print("Versand 2.95 EUR")  
else:  
    print("Versand 4.95 EUR")
```

if, elif, else und match waeren ohne Vergleichs- und Logikoperatoren stumm.

Heute geübt

- ✓ Arithmetik: + - * / // % **
- ✓ / vs. // vs. % praktisch nutzen
- ✓ Vergleiche: ==, !=, <, >, <=, >=
- ✓ Logik: and, or, not mit Wahrheitstabellen
- ✓ is vs. == – Identität vs. Gleichheit
- ✓ in / not in auf Listen und Strings
- ✓ Mengenoperationen auf Sets: |, &, -, ^

Zur Vertiefung im Notebook 08:

- “Sofort ausprobieren”-Aufgaben in Kap. 1-4 (sind Sie schon mitgegangen)
- “Grosse Uebungen” am Ende jedes Kapitels: Restaurant-Rechnung, Schaltjahr-Pruefung, kombinierte Bedingungen
- Trainingsmaterial mit Musterloesungen am Notebook-Ende